Lab 04

Simple Python Database Connections

Name	Zunaira Tariq
Registration No.	2023-BCS-095
Department	Computer Science (CS)
Course	DevOps
Submitted to	Sir Waqas
GitHub	@ZunairaTariq24
Databases fork	https://github.com/ZunairaTariq24/Databases

Overview

This lab connects Python to MySQL and MongoDB running in Docker. The same **db_connect.py** file was edited through four stages: values in code (MySQL), values in code (MongoDB), environment variables with **os.getenv()**, and an INI credentials file read with **configparser**. Each program only connects, sends a request and prints which database answered.

Fork URL: https://github.com/ZunairaTariq24/Databases

Known issue: MongoDB 8.0 and the host kernel

On this host (Linux kernel 6.19 or newer) the **mongo:8.0** containers exited immediately with the message that this kernel version has a known incompatibility with this version of MongoDB (SERVER-121912). The CPU was not the cause (AVX is supported). As a workaround, a **docker-compose.override.yml** was added that sets **GLIBC_TUNABLES=glibc.pthread.rseq=1** on mongo-dev, mongo-test and mongo-prod. No other compose settings were changed, and after this the three MongoDB services started and connected on ports 27017, 27018 and 27019.

Task 1: Fork and Clone the Databases Repository

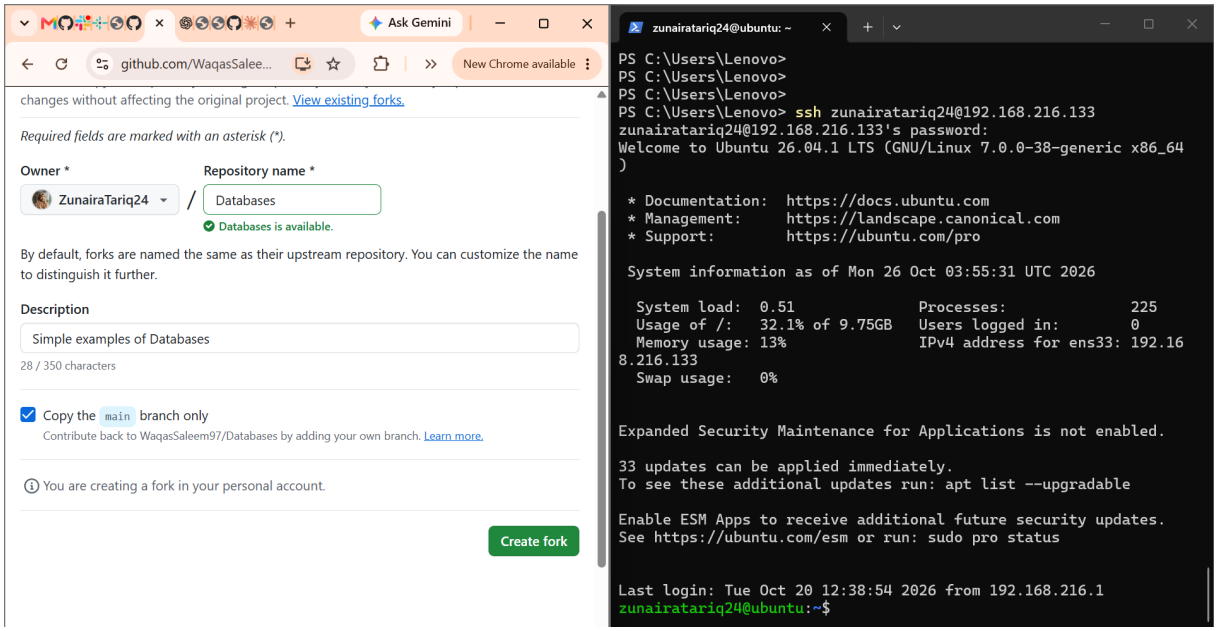


Figure 1: databases_fork.png - the fork ZunairaTariq24/Databases on GitHub.

```
git clone git@github.com:ZunairaTariq24/Databases.git
```

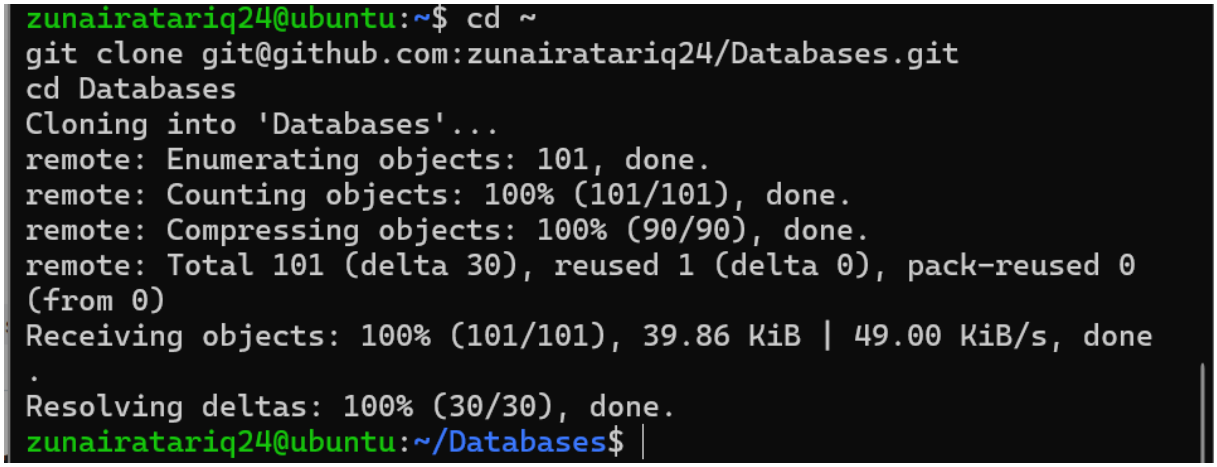


Figure 2: databases_clone.png - cloning the fork on the Ubuntu server.

Task 2: Install Docker, Docker Compose and Python

```
git --version; docker --version; docker compose version
```

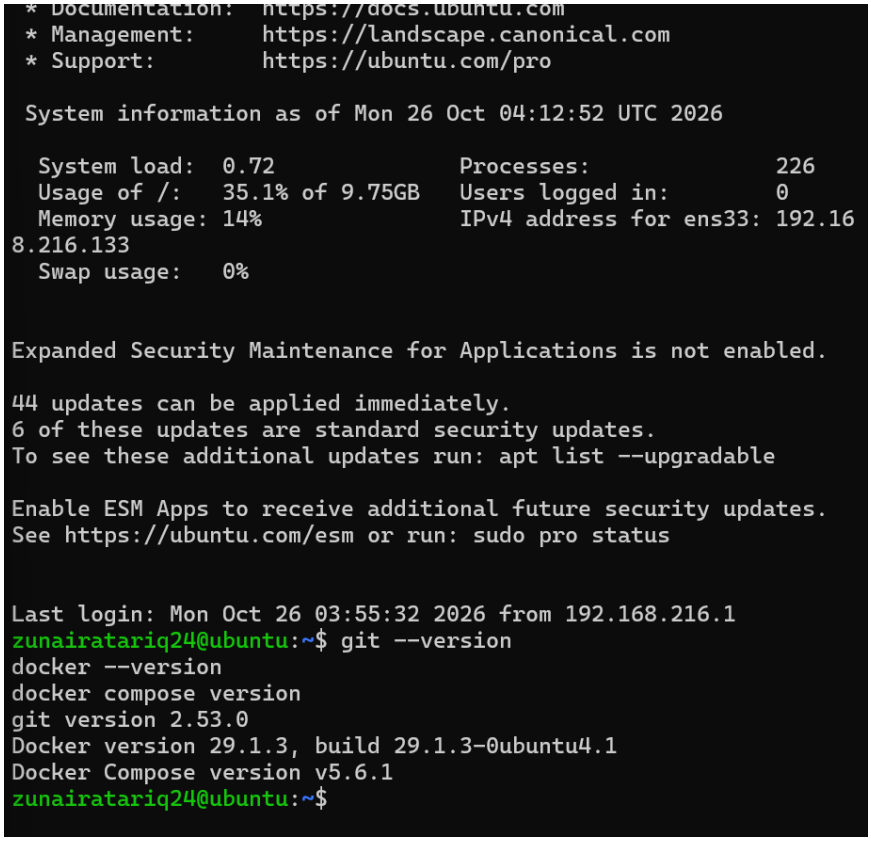


Figure 3: docker_installation.png - git, docker and docker compose versions.

Task 3: Start the Database Services

```
docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod; docker
compose ps
```

```
(.venv) zunairatariq24@ubuntu:~/Databases$ docker compose up -d
--wait mysql mongo-dev mongo-test mongo-prod
docker compose ps
[+] up 29/33
 ✔ Image mysql:8.4                    Pulled                391.9s
 ✔ Image mongo:8.0                    Pulled                548.9s
 ✔ Network databases_default          Created                 0.1s
 ✔ Volume databases_mongo-dev-data    Created                 0.0s
 ✔ Volume databases_mongo-prod-data   Created                 0.0s
 ✔ Volume databases_mongo-test-data   Created                 0.0s
 ✔ Volume databases_mysql-data        Created                 0.0s
 ⠼ Container databases-mysql-1        Waiting                 3.7s
 ⠼ Container databases-mongo-test-1   Waiting                 3.7s
 ⠼ Container databases-mongo-prod-1   Waiting                 3.7s
 ⠼ Container databases-mongo-dev-1    Waiting                 3.7s
container databases-mongo-prod-1 exited (1)
NAME                IMAGE       COMMAND                  SERVICE
   CREATED          STATUS                          PORTS
databases-mysql-1   mysql:8.4   "docker-entrypoint.s…"   mysql
   4 seconds ago    Up 3 seconds (health: starting)   33060/tcp,
127.0.0.1:3307->3306/tcp
(.venv) zunairatariq24@ubuntu:~/Databases$
```

Figure 4: database_services.png - starting the four database services.

```
docker compose port mysql 3306 (and mongo-dev / mongo-test / mongo-prod on
27017)
```

```
(.venv) zunairatariq24@ubuntu:~/Databases$ docker compose port m
ysql 3306
127.0.0.1:3307
(.venv) zunairatariq24@ubuntu:~/Databases$ docker compose port m
ongo-dev 27017
service "mongo-dev" is not running
(.venv) zunairatariq24@ubuntu:~/Databases$ docker compose port m
ongo-test 27017
service "mongo-test" is not running
(.venv) zunairatariq24@ubuntu:~/Databases$ docker compose port m
ongo-prod 27017
service "mongo-prod" is not running
(.venv) zunairatariq24@ubuntu:~/Databases$
```

Figure 5: database_ports.png - host ports of the database services.

Task 4: MySQL with Values in the Code

```
python db_connect.py
```

```
(.venv) zunairatariq24@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) zunairatariq24@ubuntu:~/Databases$
```

Figure 6: mysql_hardcoded_connection.png - connected to school_mysql on 127.0.0.1:3307, environment dev.

Task 5: MongoDB with Values in the Code

```
python db_connect.py
```

```
(.venv) zunairatariq24@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) zunairatariq24@ubuntu:~/Databases$
```

Figure 7: mongo_hardcoded_connection.png - connected to school_dev on port 27017, environment dev.

Task 6: MongoDB with os.getenv()

The same program reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT and MONGO_DATABASE from the shell environment, so one unchanged file reaches three databases.



Figure 8: mongo_env_dev.png - school_dev, port 27017, environment dev.

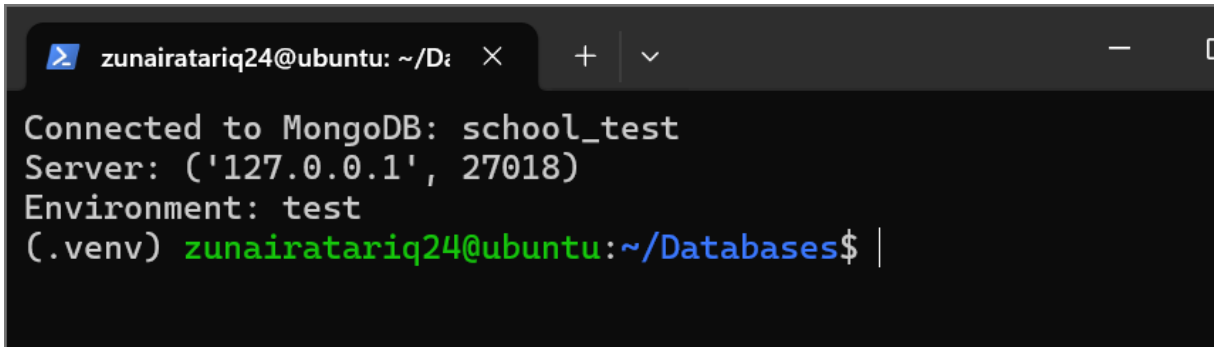


Figure 9: mongo_env_test.png - school_test, port 27018, environment test.

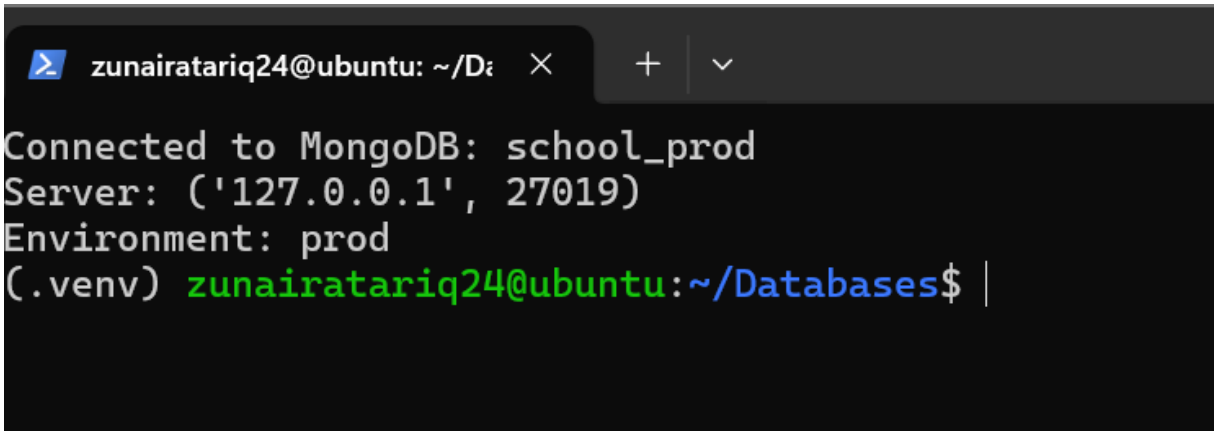


Figure 10: mongo_env_prod.png - school_prod, port 27019, environment prod.

Task 7: MongoDB with a Credentials File

```
cp -n credentials.ini.example credentials.ini; ls credentials.ini
```

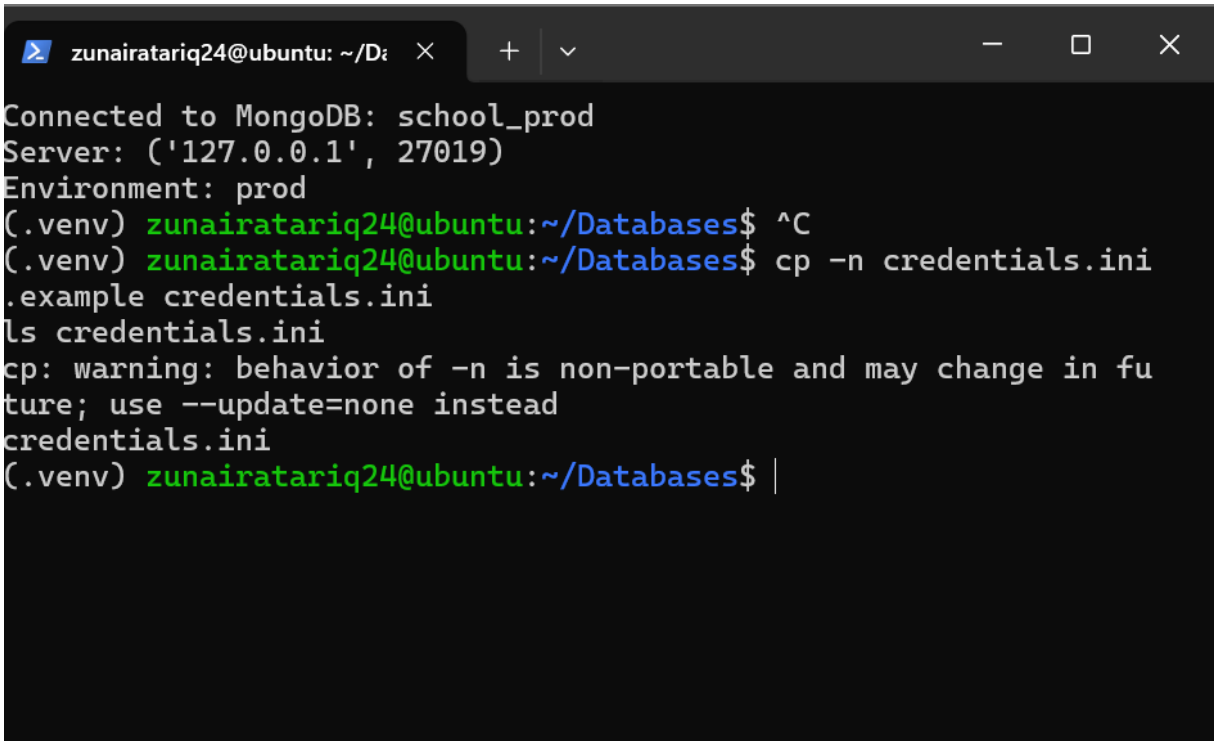


Figure 11: credential_files_created.png - credentials.ini created from credentials.ini.example.

```
chmod 600 credentials.ini; stat -c '%a %n' credentials.ini
```

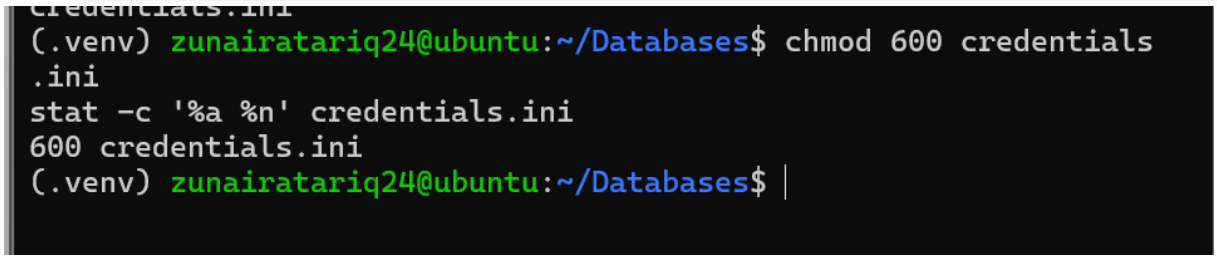


Figure 12: credential_file_permissions.png - file restricted to the owner (mode 600).

The program reads APP_ENV (default dev) to choose a section of credentials.ini.

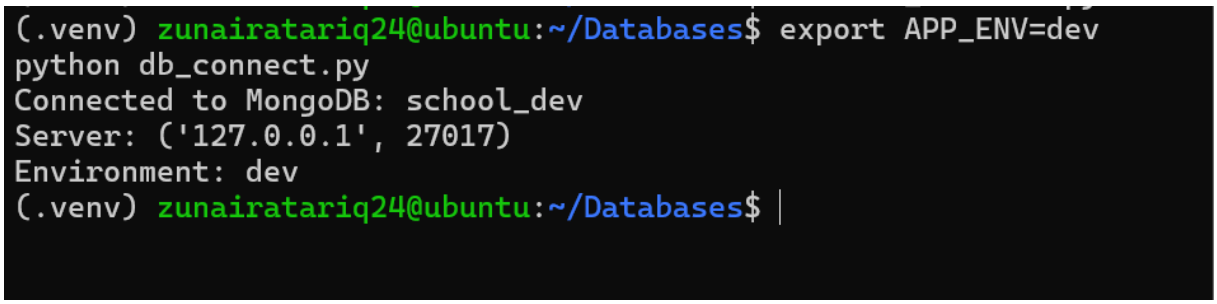


Figure 13: mongo_file_dev.png - APP_ENV=dev: school_dev, port 27017.

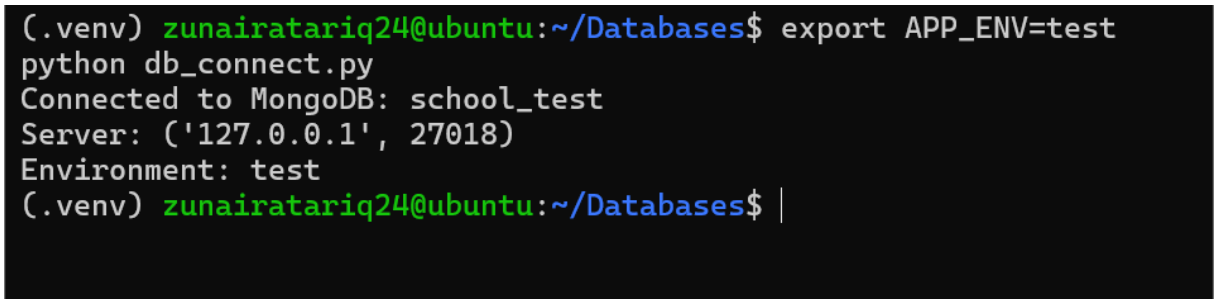


Figure 14: mongo_file_test.png - APP_ENV=test: school_test, port 27018.

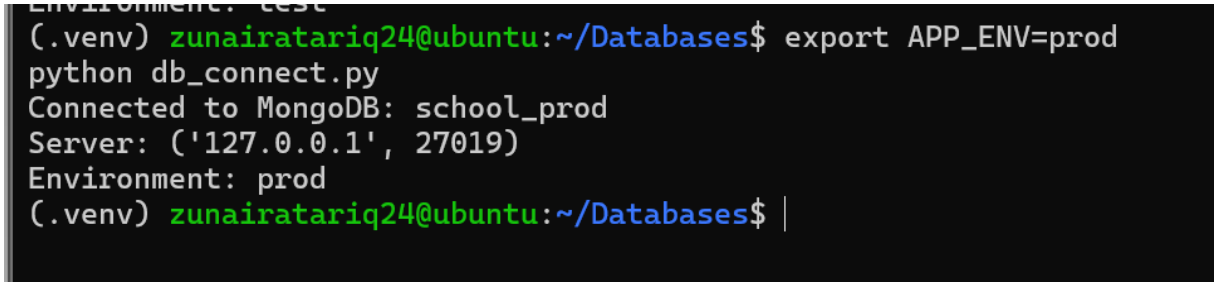


Figure 15: mongo_file_prod.png - APP_ENV=prod: school_prod, port 27019.

Exam Evaluation Questions

1. A missing environment variable

After **unset MONGO_DATABASE**, the call **os.getenv("MONGO_DATABASE")** returns **None** because the variable no longer exists in the shell environment. The statement **if not username or not password or not endpoint or not database_name** then evaluates to true, so the program raises **SystemExit** with the message asking to set all four variables. It stops before the URI is built or any connection is attempted. After restoring **export MONGO_DATABASE=school_prod**, all four values are present and the program connects again.

2. Which source does the file version read?

The file version does not read MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT or MONGO_DATABASE at all. It reads only **APP_ENV** to choose a section of credentials.ini, and every connection setting comes from that section. With **APP_ENV** unset, **os.getenv("APP_ENV", "dev")** falls back to the default **dev**, so it connects to the dev database even though the shell still holds the prod values. Setting **APP_ENV=test** selects the [test] section and connects to the test database.

3. A wrong password in the file

The endpoint only tells the client where the server is; it does not prove who the client is. MongoDB checks the username and password against the user stored for that database, so with **password = wrong** in the [test] section the server is reached but rejects the login with an authentication error. A correct endpoint is therefore not enough: the credentials must match too. After restoring the supplied test password the connection works again, and credentials.ini is never committed.